# 8051 REGISTER BANKS AND STACK

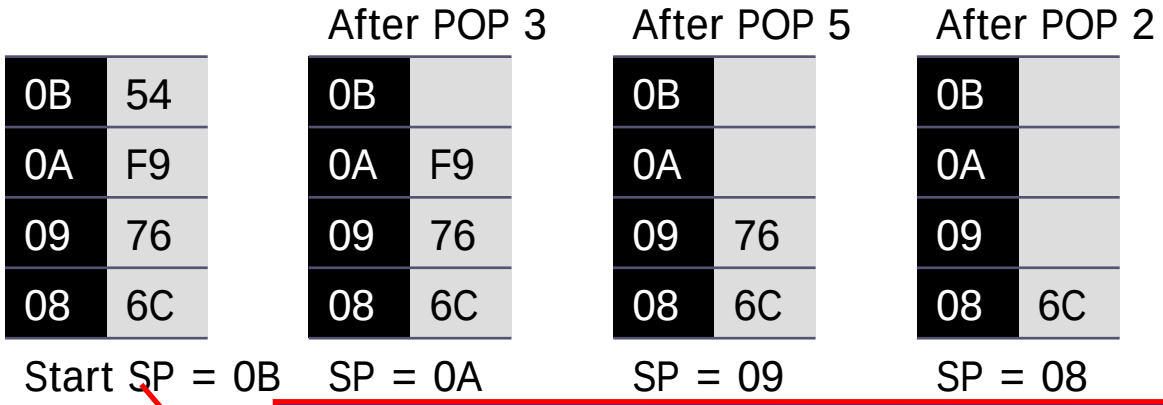
## Popping From Stack

Example 2-9

Examining the stack, show the contents of the register and SP after execution of the following instructions. All value are in hex.

```
POP      3          ; POP stack into R3
POP      5          ; POP stack into R5
POP      2          ; POP stack into R2
```

**Solution:**




Because locations 20-2FH of RAM are reserved for bit-addressable memory, so we can change the SP to other RAM location by using the instruction "MOV SP, #XX"

8051 REGISTER BANKS AND STACK

CALL Instruction And Stack

- The CPU also uses the stack to save the address of the instruction just below the CALL instruction
  - This is how the CPU knows where to resume when it returns from the called subroutine

8051 REGISTER BANKS AND STACK

Incrementing Stack Pointer

- The reason of incrementing SP after push is
  - Make sure that the stack is growing toward RAM location 7FH, from lower to upper addresses
  - Ensure that the stack will not reach the bottom of RAM and consequently run out of stack space
  - If the stack pointer were decremented after push
    - We would be using RAM locations 7, 6, 5, etc. which belong to R7 to R0 of bank 0, the default register bank

8051 REGISTER BANKS AND STACK

Stack and Bank 1 Conflict

- When 8051 is powered up, register bank 1 and the stack are using the same memory space
  - We can reallocate another section of RAM to the stack



# 8051 REGISTER BANKS AND STACK

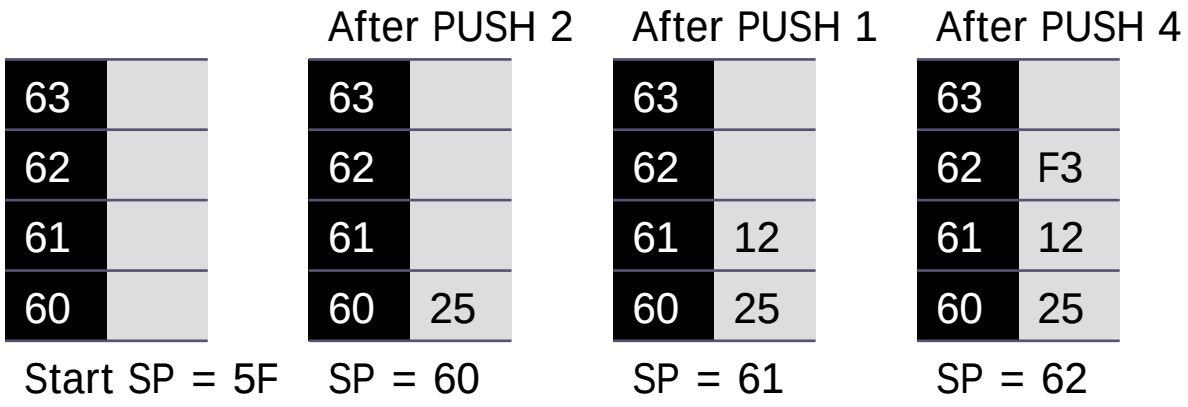
## Stack And Bank 1 Conflict (cont')

Example 2-10

Examining the stack, show the contents of the register and SP after execution of the following instructions. All value are in hex.

```
MOV SP, #5FH    ;make RAM location 60H
                ;first stack location
MOV R2, #25H
MOV R1, #12H
MOV R4, #0F3H
PUSH  2
PUSH  1
PUSH  4
```

**Solution:**



# JUMP, LOOP AND CALL INSTRUCTIONS

*The 8051 Microcontroller and Embedded Systems: Using Assembly and C*
Mazidi, Mazidi and McKinlay

Chung-Ping Young
楊中平



***Home Automation, Networking, and Entertainment Lab***

Dept. of Computer Science and Information Engineering
National Cheng Kung University, TAIWAN

# LOOP AND JUMP INSTRUCTIONS

## Looping

- Repeating a sequence of instructions a certain number of times is called a *loop*
  - Loop action is performed by
    `DJNZ reg, Label`
    - The register is decremented
    - If it is not zero, it jumps to the target address referred to by the label
    - Prior to the start of loop the register is loaded with the counter for the number of repetitions
    - Counter can be R0 – R7 or RAM location

A loop can be repeated a maximum of 255 times, if R2 is FFH

```
;This program adds value 3 to the ACC ten times
        MOV  A,#0      ;A=0, clear ACC
        MOV  R2,#10    ;load counter R2=10
AGAIN:  ADD  A,#03     ;add 03 to ACC
        DJNZ R2,AGAIN  ;repeat until R2=0,10 times
        MOV  R5,A      ;save A in R5
```

LOOP AND JUMP INSTRUCTIONS

Nested Loop

- If we want to repeat an action more times than 256, we use a loop inside a loop, which is called *nested loop*
  - We use multiple registers to hold the count

Write a program to (a) load the accumulator with the value 55H, and (b) complement the ACC 700 times

```
        MOV   A,#55H    ;A=55H
        MOV   R3,#10    ;R3=10, outer loop count
NEXT:   MOV   R2,#70    ;R2=70, inner loop count
AGAIN:  CPL   A         ;complement A register
        DJNZ  R2,AGAIN  ;repeat it 70 times
        DJNZ  R3,NEXT
```

LOOP AND JUMP INSTRUCTIONS

Conditional Jumps

- Jump only if a certain condition is met

**JZ label ;jump if A=0**

```
        MOV   A,R0      ;A=R0
        JZ    OVER      ;jump if A = 0
        MOV   A,R1      ;A=R1
        JZ    OVER      ;jump if A = 0
        ...
OVER:
```

Can be used only for register A, not any other register

Determine if R5 contains the value 0. If so, put 55H in it.

```
        MOV   A,R5      ;copy R5 to A
        JNZ   NEXT      ;jump if A is not zero
        MOV   R5,#55H
NEXT:   ...
```

LOOP AND JUMP INSTRUCTIONS

Conditional Jumps (cont')

- (cont')

**JNC label ;jump if no carry, CY=0**

  - If CY = 0, the CPU starts to fetch and execute instruction from the address of the label
  - If CY = 1, it will not jump but will execute the next instruction below JNC

Find the sum of the values 79H, F5H, E2H. Put the sum in registers R0 (low byte) and R5 (high byte).

```
        MOV   A,#0       ;A=0
        MOV   R5,A       ;clear R5
        ADD   A,#79H     ;A=0+79H=79H
;       JNC   N_1        ;if CY=0, add next number
;       INC   R5         ;if CY=1, increment R5
N_1:    ADD   A,#0F5H    ;A=79+F5=6E and CY=1
        JNC   N_2        ;jump if CY=0
        INC   R5        ;if CY=1,increment R5 (R5=1)
N_2:    ADD   A,#0E2H    ;A=6E+E2=50 and CY=1
        JNC   OVER       ;jump if CY=0
        INC   R5         ;if CY=1, increment 5
OVER:   MOV   R0,A       ;now R0=50H, and R5=02
```

MOV R5,#0



# LOOP AND JUMP INSTRUCTIONS

## Conditional Jumps (cont')

8051 conditional jump instructions

| Instructions | Actions |
|---|---|
| JZ | Jump if A = 0 |
| JNZ | Jump if A ≠ 0 |
| DJNZ | Decrement and Jump if A ≠ 0 |
| CJNE A,byte | Jump if A ≠ byte |
| CJNE reg,#data | Jump if byte ≠ #data |
| JC | Jump if CY = 1 |
| JNC | Jump if CY = 0 |
| JB | Jump if bit = 1 |
| JNB | Jump if bit = 0 |
| JBC | Jump if bit = 1 and clear bit |

- All conditional jumps are short jumps
  - The address of the target must within -128 to +127 bytes of the contents of PC

LOOP AND JUMP INSTRUCTIONS

Unconditional Jumps

- The unconditional jump is a jump in which control is transferred unconditionally to the target location

**LJMP** (long jump)

- 3-byte instruction
  - First byte is the opcode
  - Second and third bytes represent the 16-bit target address
    - Any memory location from 0000 to FFFFH

**SJMP** (short jump)

- 2-byte instruction
  - First byte is the opcode
  - Second byte is the relative target address
    - 00 to FFH (forward +127 and backward -128 bytes from the current PC)

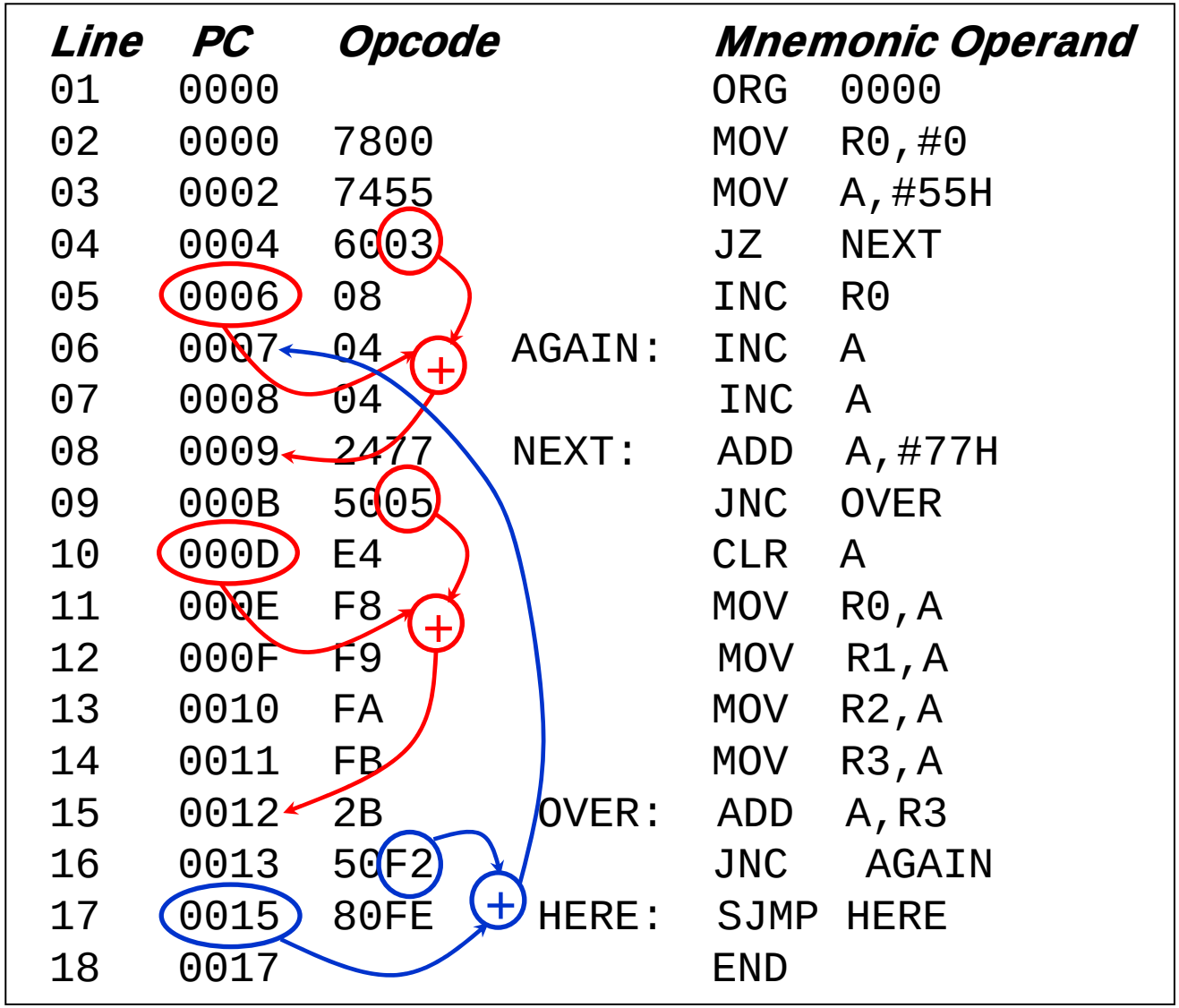
## LOOP AND JUMP INSTRUCTIONS

## Calculating Short Jump Address

- To calculate the target address of a short jump (`SJMP`, `JNC`, `JZ`, `DJNZ`, etc.)
  - The second byte is added to the PC of the instruction immediately below the jump
- If the target address is more than -128 to +127 bytes from the address below the short jump instruction
  - The assembler will generate an error stating the jump is out of range

# LOOP AND JUMP INSTRUCTIONS

## Calculating Short Jump Address (cont')



| *Line* | *PC* | *Opcode* | | *Mnemonic Operand* |
|---|---|---|---|---|
| 01 | 0000 | | | ORG 0000 |
| 02 | 0000 | 7800 | | MOV R0,#0 |
| 03 | 0002 | 7455 | | MOV A,#55H |
| 04 | 0004 | 6003 | | JZ NEXT |
| 05 | 0006 | 08 | | INC R0 |
| 06 | 0007 | 04 | AGAIN: | INC A |
| 07 | 0008 | 04 | | INC A |
| 08 | 0009 | 2477 | NEXT: | ADD A,#77H |
| 09 | 000B | 5005 | | JNC OVER |
| 10 | 000D | E4 | | CLR A |
| 11 | 000E | F8 | | MOV R0,A |
| 12 | 000F | F9 | | MOV R1,A |
| 13 | 0010 | FA | | MOV R2,A |
| 14 | 0011 | FB | | MOV R3,A |
| 15 | 0012 | 2B | OVER: | ADD A,R3 |
| 16 | 0013 | 50F2 | | JNC AGAIN |
| 17 | 0015 | 80FE | HERE: | SJMP HERE |
| 18 | 0017 | | | END |

# CALL INSTRUCTIONS

- Call instruction is used to call subroutine
  - Subroutines are often used to perform tasks that need to be performed frequently
  - This makes a program more structured in addition to saving memory space

**LCALL** (long call)

- 3-byte instruction
  - First byte is the opcode
  - Second and third bytes are used for address of target subroutine
    - Subroutine is located anywhere within 64K byte address space

**ACALL** (absolute call)

- 2-byte instruction
  - 11 bits are used for address within 2K-byte range

# CALL INSTRUCTIONS

## LCALL

- When a subroutine is called, control is transferred to that subroutine, the processor
  - Saves on the stack the the address of the instruction immediately below the LCALL
  - Begins to fetch instructions form the new location
- After finishing execution of the subroutine
  - The instruction RET transfers control back to the caller
    - Every subroutine needs RET as the last instruction

# CALL INSTRUCTIONS

## LCALL (cont')

```
        ORG    0
BACK:   MOV    A,#55H     ;load A with 55H
        MOV    P1,A       ;send 55H to port 1
        LCALL  DELAY      ;time delay
        MOV    A,#0AAH    ;load A with AA (in hex)
        MOV    P1,A       ;send AAH to port 1
        LCALL  DELAY
        SJMP   BACK       ;keep doing this indefinitely


;---------- this is delay subroutine ------------
        ORG    300H       ;put DELAY at address 300H
DELAY:  MOV    R5,#0FFH   ;R5=255 (FF in hex), counter
AGAIN:  DJNZ   R5,AGAIN   ;stay here until R5 become 0
        RET               ;return to caller (when R5 =0)
        END
```

Upon executing "LCALL DELAY", the address of instruction below it, "MOV A,#0AAH" is pushed onto stack, and the 8051 starts to execute at 300H.

The counter R5 is set to FFH; so loop is repeated 255 times.

When R5 becomes 0, control falls to the RET which pops the address from the stack into the PC and resumes executing the instructions after the CALL.

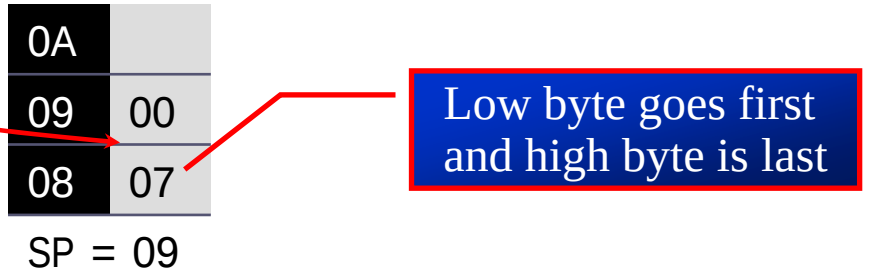
The amount of time delay depends on the frequency of the 8051

# CALL INSTRUCTIONS

## CALL Instruction and Stack

```
001 0000                ORG  0
002 0000 7455   BACK:   MOV  A,#55H  ;load A with 55H
003 0002 F590           MOV  P1,A    ;send 55H to p1
004 0004 120300         LCALL DELAY  ;time delay
005 0007 74AA           MOV  A,#0AAH ;load A with AAH
006 0009 F590           MOV  P1,A    ;send AAH to p1
007 000B 120300         LCALL DELAY
008 000E 80F0           SJMP  BACK   ;keep doing this
009 0010
010 0010 ;-------this is the delay subroutine------
011 0300                ORG  300H
012 0300        DELAY:
013 0300 7DFF           MOV  R5,#0FFH ;R5=255
014 0302 DDFE   AGAIN:  DJNZ R5,AGAIN ;stay here
015 0304 22             RET           ;return to caller
016 0305                END            ;end of asm file
```

Stack frame after the first LCALL



# CALL INSTRUCTIONS

## Use PUSH/POP in Subroutine

```
01 0000                    ORG   0
02 0000 7455    BACK:      MOV   A,#55H   ;load A with 55H
03 0002 F590               MOV   P1,A     ;send 55H to p1
04 0004 7C99               MOV   R4,#99H
05 0006 7D67               MOV   R5,#67H
06 0008 120300             LCALL DELAY    ;time delay
07 000B 74AA               MOV   A,#0AAH  ;load A with AA
08 000D F590               MOV   P1,A     ;send AAH to p1
09 000F 120300             LCALL DELAY
10 0012 80EC               SJMP  BACK     ;keeping doing
   this
11 0014 ;-------this is the delay subroutine------
12 0300                    ORG   300H
13 0300 C004    DELAY:     PUSH  4        ;push R4
14 0302 C005               PUSH  5        ;push R5
   0304 7CFF               MOV   R4,#0FFH;R4=FFH
   0306 7DFF    NEXT:      MOV   R5,#0FFH;R5=FFH
   0308 DDFE    AGAIN:     DJNZ  R5,AGAIN
   030A DCFA               DJNZ  R4,NEXT
   030C D005               POP   5        ;POP into R5
   030E D004               POP   4        ;POP into R4
   031                     RET            ;return to caller
22 031                                    e
```

Normally, the number of PUSH and POP instructions must always match in any called subroutine

| After first LCALL | | | After PUSH 4 | | | After PUSH 5 | | |
|---|---|---|---|---|---|---|---|---|
| 0B | | | 0B | | | 0B | 67 | R5 |
| 0A | | | 0A | 99 | R4 | 0A | 99 | R4 |
| 09 | 00 | PCH | 09 | 00 | PCH | 09 | 00 | PCH |
| 08 | 0B | PCL | 08 | 0B | PCL | 08 | 0B | PCL |

# CALL INSTRUCTIONS

## Calling Subroutines

```
;MAIN program calling subroutines
        ORG  0
MAIN:   LCALL           SUBR_1
        LCALL           SUBR_2
        LCALL           SUBR_3

HERE:   SJMP            HERE
;-----------end of MAIN

SUBR_1: ...
        ...
        RET
;-----------end of subroutine1

SUBR_2: ...
        ...
        RET
;-----------end of subroutine2

SUBR_3: ...
        ...
        RET
;-----------end of subroutine3
        END             ;end of the asm file
```

It is common to have one main program and many subroutines that are called from the main program

This allows you to make each subroutine into a separate module

- Each module can be tested separately and then brought together with main program

- In a large program, the module can be assigned to different programmers

CALL INSTRUCTIONS

ACALL

- The only difference between ACALL and LCALL is
  - The target address for LCALL can be anywhere within the 64K byte address
  - The target address of ACALL must be within a 2K-byte range
- The use of ACALL instead of LCALL can save a number of bytes of program ROM space

# CALL INSTRUCTIONS

## ACALL (cont')

```
        ORG   0
BACK:   MOV   A,#55H    ;load A with 55H
        MOV   P1,A      ;send 55H to port 1
        LCALL DELAY     ;time delay
        MOV   A,#0AAH   ;load A with AA (in hex)
        MOV   P1,A      ;send AAH to port 1
        LCALL DELAY
        SJMP  BACK      ;keep doing this indefinitely
         ...
        END             ;end of asm file
```

A rewritten program which is more efficiently

```
        ORG   0
        MOV   A,#55H    ;load A with 55H
BACK:   MOV   P1,A      ;send 55H to port 1
        ACALL DELAY     ;time delay
        CPL   A         ;complement reg A
        SJMP  BACK      ;keep doing this indefinitely
         ...
        END             ;end of asm file
```

# TIME DELAY FOR VARIOUS 8051 CHIPS

- CPU executing an instruction takes a certain number of clock cycles
  - These are referred as to as *machine cycles*
- The length of machine cycle depends on the frequency of the crystal oscillator connected to 8051
- In original 8051, one machine cycle lasts 12 oscillator periods

Find the period of the machine cycle for 11.0592 MHz crystal frequency

**Solution:**

```
11.0592/12 = 921.6 kHz;
   machine cycle is 1/921.6 kHz = 1.085 μs
```

# TIME DELAY FOR VARIOUS 8051 CHIPS (cont')

For 8051 system of 11.0592 MHz, find how long it takes to execute each instruction.

```
(a) MOV R3,#55 (b) DEC R3 (c) DJNZ R2 target
(d) LJMP (e) SJMP (f) NOP (g) MUL AB
```

**Solution:**

| | *Machine cycles* | *Time to execute* |
|---|---|---|
| (a) | 1 | 1x1.085 $\mu$s = 1.085 $\mu$s |
| (b) | 1 | 1x1.085 $\mu$s = 1.085 $\mu$s |
| (c) | 2 | 2x1.085 $\mu$s = 2.17 $\mu$s |
| (d) | 2 | 2x1.085 $\mu$s = 2.17 $\mu$s |
| (e) | 2 | 2x1.085 $\mu$s = 2.17 $\mu$s |
| (f) | 1 | 1x1.085 $\mu$s = 1.085 $\mu$s |
| (g) | 4 | 4x1.085 $\mu$s = 4.34 $\mu$s |

# TIME DELAY FOR VARIOUS 8051 CHIPS

## Delay Calculation

Find the size of the delay in following program, if the crystal frequency is 11.0592MHz.

```
        MOV  A,#55H
AGAIN:  MOV  P1,A
        ACALL DELAY
        CPL  A
        SJMP AGAIN
;---time delay-------
DELAY:  MOV  R3,#200
HERE:   DJNZ R3,HERE
        RET
```

A simple way to short jump to itself in order to keep the microcontroller busy

```
HERE: SJMP HERE
```

We can use the following:

```
SJMP $
```

**Solution:**

| | | ***Machine cycle*** |
|---|---|---|
| DELAY: | MOV R3,#200 | 1 |
| HERE: | DJNZ R3,HERE | 2 |
| | RET | 2 |

Therefore, [(200x2)+1+2]x1.085$\mu$s = 436.255$\mu$s.

# TIME DELAY FOR VARIOUS 8051 CHIPS

## Increasing Delay Using NOP

Find the size of the delay in following program, if the crystal frequency is 11.0592MHz.

```
                          Machine Cycle
DELAY: MOV  R3,#250          1
HERE:  NOP                   1
       NOP                   1
       NOP                   1
       NOP                   1
       DJNZ R3,HERE          2
       RET                   2
```

**Solution:**

The time delay inside HERE loop is [250(1+1+1+1+2)]x1.085$\mu$s = 1627.5$\mu$s.
Adding the two instructions outside loop we have 1627.5$\mu$s + 3 x 1.085$\mu$s = 1630.755$\mu$s

# TIME DELAY FOR VARIOUS 8051 CHIPS

## Large Delay Using Nested Loop

Find the size of the delay in following program, if the crystal frequency is 11.0592MHz.

```
                          Machine Cycle
DELAY: MOV  R2,#200            1
AGAIN: MOV  R3,#250            1
HERE:  NOP                     1
       NOP                     1
       DJNZ R3,HERE            2
       DJNZ R2,AGAIN           2
       RET                     2
```

Notice in nested loop, as in all other time delay loops, the time is approximate since we have ignored the first and last instructions in the subroutine.

**Solution:**

For HERE loop, we have (4x250)x1.085$\mu$s=1085$\mu$s. For AGAIN loop repeats HERE loop 200 times, so we have 200x1085$\mu$s=217000$\mu$s. But "MOV R3,#250" and "DJNZ R2,AGAIN" at the start and end of the AGAIN loop add (3x200x1.805)=651$\mu$s. As a result we have 217000+651=217651$\mu$s.

TIME DELAY FOR VARIOUS 8051 CHIPS

Delay Calculation for Other 8051

- Two factors can affect the accuracy of the delay
  - Crystal frequency
    - The duration of the clock period of the machine cycle is a function of this crystal frequency
  - 8051 design
    - The original machine cycle duration was set at 12 clocks
    - Advances in both IC technology and CPU design in recent years have made the 1-clock machine cycle a common feature

Clocks per machine cycle for various 8051 versions

| Chip / Maker | Clocks per Machine Cycle |
|---|---|
| AT89C51 Atmel | 12 |
| P89C54X2 Philips | 6 |
| DS5000 Dallas Semi | 4 |
| DS89C420/30/40/50 Dallas Semi | 1 |

# TIME DELAY FOR VARIOUS 8051 CHIPS

## Delay Calculation for Other 8051 (cont')

Find the period of the machine cycle (MC) for various versions of 8051, if XTAL=11.0592 MHz.
(a) AT89C51 (b) P89C54X2 (c) DS5000 (d) DS89C4x0

**Solution:**

(a) 11.0592MHz/12 = 921.6kHz;
MC is 1/921.6kHz = 1.085$\mu$s = 1085ns
(b) 11.0592MHz/6 = 1.8432MHz;
MC is 1/1.8432MHz = 0.5425$\mu$s = 542ns
(c) 11.0592MHz/4 = 2.7648MHz ;
MC is 1/2.7648MHz = 0.36$\mu$s = 360ns
(d) 11.0592MHz/1 = 11.0592MHz;
MC is 1/11.0592MHz = 0.0904$\mu$s = 90ns

# TIME DELAY FOR VARIOUS 8051 CHIPS

## Delay Calculation for Other 8051 (cont')

| Instruction | 8051 | DSC89C4x0 |
|---|---|---|
| MOV R3,#55 | 1 | 2 |
| DEC R3 | 1 | 1 |
| DJNZ R2 target | 2 | 4 |
| LJMP | 2 | 3 |
| SJMP | 2 | 3 |
| NOP | 1 | 1 |
| MUL AB | 4 | 9 |

For an AT8051 and DSC89C4x0 system of 11.0592 MHz, find how long it takes to execute each instruction.

(a) MOV R3,#55 (b) DEC R3 (c) DJNZ R2 target
(d) LJMP (e) SJMP (f) NOP (g) MUL AB

**Solution:**

| | ***AT8051*** | ***DS89C4x0*** |
|---|---|---|
| (a) | 1×1085ns = 1085ns | 2×90ns = 180ns |
| (b) | 1×1085ns = 1085ns | 1×90ns = 90ns |
| (c) | 2×1085ns = 2170ns | 4×90ns = 360ns |
| (d) | 2×1085ns = 2170ns | 3×90ns = 270ns |
| (e) | 2×1085ns = 2170ns | 3×90ns = 270ns |
| (f) | 1×1085ns = 1085ns | 1×90ns = 90ns |
| (g) | 4×1085ns = 4340ns | 9×90ns = 810ns |

# I/O PORT PROGRAMMING

*The 8051 Microcontroller and Embedded Systems: Using Assembly and C*
Mazidi, Mazidi and McKinlay

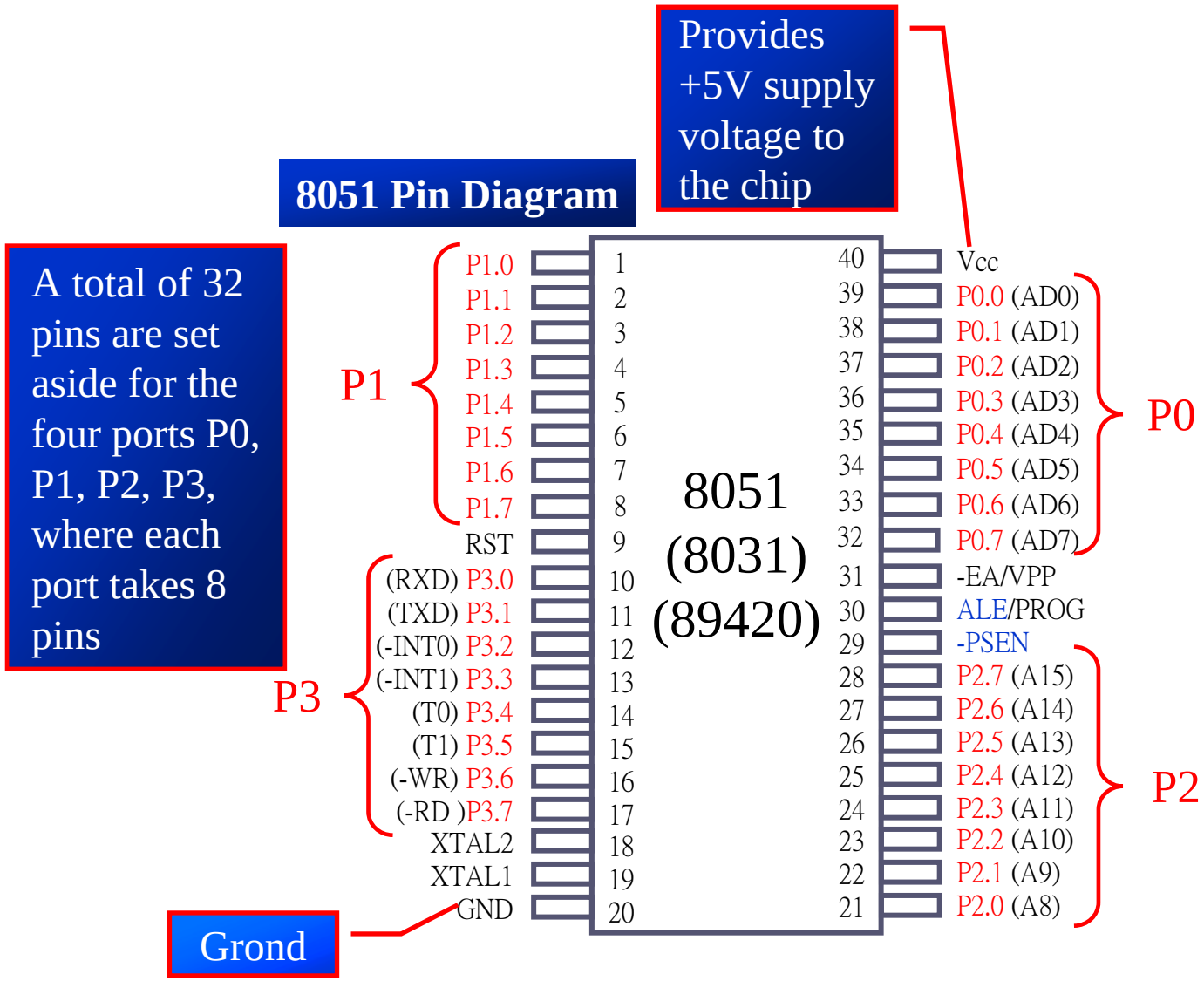
Chung-Ping Young
楊中平



***Home Automation, Networking, and Entertainment Lab***

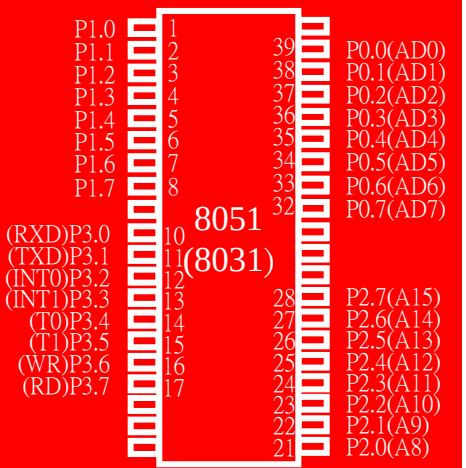
Dept. of Computer Science and Information Engineering
National Cheng Kung University, TAIWAN

# I/O PROGRAMMING



# I/O PROGRAMMING

## I/O Port Pins

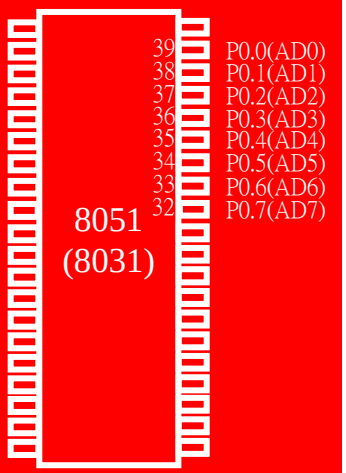


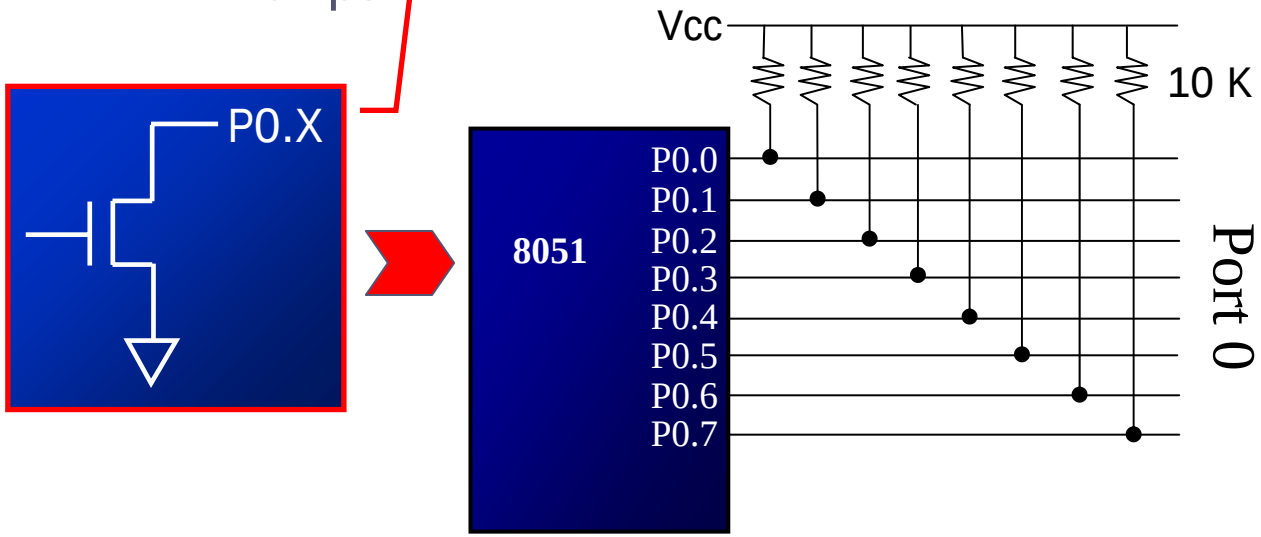

- The four 8-bit I/O ports P0, P1, P2 and P3 each uses 8 pins
- All the ports upon RESET are configured as input, ready to be used as input ports
  - When the first 0 is written to a port, it becomes an output
  - To reconfigure it as an input, a 1 must be sent to the port
    - To use any of these ports as an input port, it must be programmed

# I/O PROGRAMMING

## Port 0

- It can be used for input or output, each pin must be connected externally to a 10K ohm pull-up resistor
  - This is due to the fact that P0 is an open drain, unlike P1, P2, and P3
    - *Open drain* is a term used for MOS chips in the same way that open collector is used for TTL chips



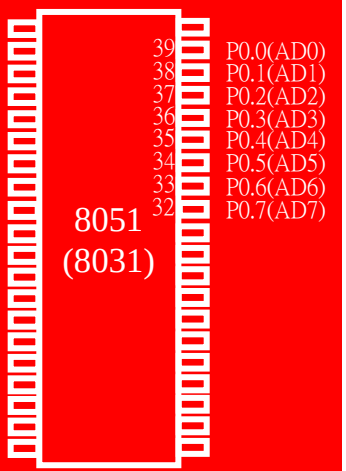



# I/O PROGRAMMING

## Port 0 (cont')

The following code will continuously send out to port 0 the alternating value 55H and AAH

```
BACK:   MOV     A,#55H
        MOV     P0,A
        ACALL   DELAY
        MOV     A,#0AAH
        MOV     P0,A
        ACALL   DELAY
        SJMP    BACK
```



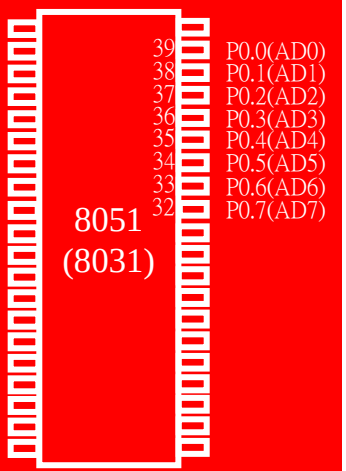
# I/O PROGRAMMING

## Port 0 as Input

- In order to make port 0 an input, the port must be programmed by writing 1 to all the bits

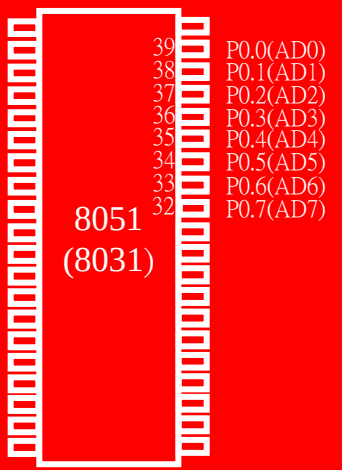
Port 0 is configured first as an input port by writing 1s to it, and then data is received from that port and sent to P1

```
        MOV     A,#0FFH         ;A=FF hex
        MOV     P0,A            ;make P0 an i/p port
                                ;by writing it all 1s
BACK:   MOV     A,P0            ;get data from P0
        MOV     P1,A            ;send it to port 1
        SJMP    BACK            ;keep doing it
```



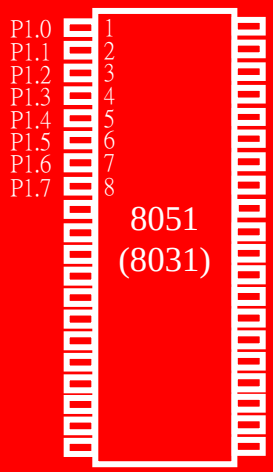
## Dual Role of Port 0

- Port 0 is also designated as AD0-AD7, allowing it to be used for both address and data
  - When connecting an 8051/31 to an external memory, port 0 provides both address and data



# I/O PROGRAMMING

## Port 1



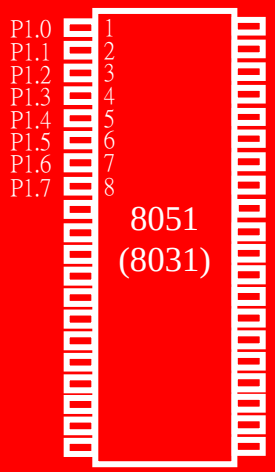

- Port 1 can be used as input or output
  - In contrast to port 0, this port does not need any pull-up resistors since it already has pull-up resistors internally
  - Upon reset, port 1 is configured as an input port

The following code will continuously send out to port 0 the alternating value 55H and AAH

```
        MOV     A,#55H
BACK:   MOV     P1,A
        ACALL   DELAY
        CPL     A
        SJMP    BACK
```

# I/O PROGRAMMING

## Port 1 as Input




- To make port 1 an input port, it must be programmed as such by writing 1 to all its bits

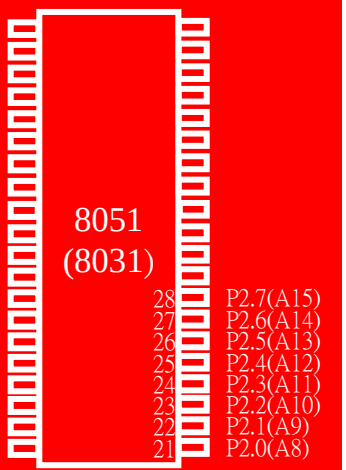
Port 1 is configured first as an input port by writing 1s to it, then data is received from that port and saved in R7 and R5

```
        MOV     A,#0FFH     ;A=FF hex
        MOV     P1,A        ;make P1 an input port
                            ;by writing it all 1s
        MOV     A,P1        ;get data from P1
        MOV     R7,A        ;save it to in reg R7
        ACALL   DELAY       ;wait
        MOV     A,P1        ;another data from P1
        MOV     R5,A        ;save it to in reg R5
```

# I/O PROGRAMMING

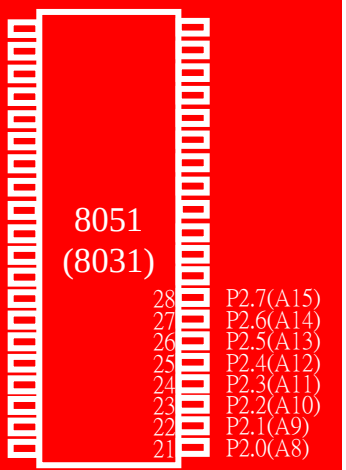
## Port 2

- Port 2 can be used as input or output
  - Just like P1, port 2 does not need any pull-up resistors since it already has pull-up resistors internally
  - Upon reset, port 2 is configured as an input port



# I/O PROGRAMMING

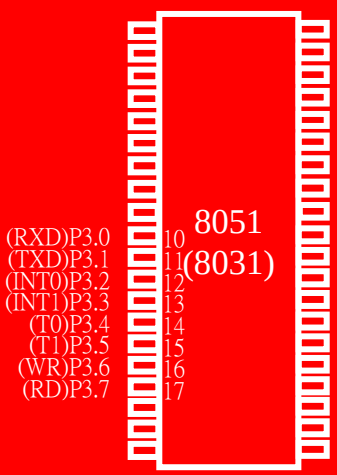
## Port 2 as Input or Dual Role




- To make port 2 an input port, it must be programmed as such by writing 1 to all its bits
- In many 8051-based system, P2 is used as simple I/O
- In 8031-based systems, port 2 must be used along with P0 to provide the 16-bit address for the external memory
  - Port 2 is also designated as A8 – A15, indicating its dual function
  - Port 0 provides the lower 8 bits via A0 – A7

# I/O PROGRAMMING

## Port 3




- Port 3 can be used as input or output
  - Port 3 does not need any pull-up resistors
  - Port 3 is configured as an input port upon reset, this is not the way it is most commonly used

# Port 3 (cont')

- Port 3 has the additional function of providing some extremely important signals

| P3 Bit | Function | Pin |
|---|---|---|
| P3.0 | RxD | 10 |
| P3.1 | TxD | 11 |
| P3.2 | $\overline{\text{INT0}}$ | 12 |
| P3.3 | $\overline{\text{INT1}}$ | 13 |
| P3.4 | T0 | 14 |
| P3.5 | T1 | 15 |
| P3.6 | $\overline{\text{WR}}$ | 16 |
| P3.7 | $\overline{\text{RD}}$ | 17 |

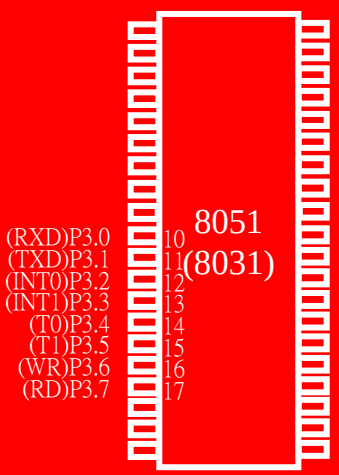
Serial communications

External interrupts

Timers

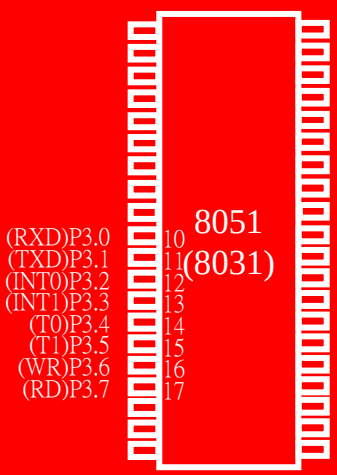
Read/Write signals of external memories

In systems based on 8751, 89C51 or DS89C4x0, pins 3.6 and 3.7 are used for I/O while the rest of the pins in port 3 are normally used in the alternate function role



# I/O PROGRAMMING

## Port 3 (cont')




Write a program for the DS89C420 to toggle all the bits of P0, P1, and P2 every 1/4 of a second

```
        ORG     0
BACK:   MOV     A,#55H
        MOV     P0,A
        MOV     P1,A
        MOV     P2,A
        ACALL   QSDELAY     ;Quarter of a second
        MOV     A,#0AAH
        MOV     P0,A
        MOV     P1,A
        MOV     P2,A
        ACALL   QSDELAY
        SJMP    BACK
QSDELAY:
        MOV     R5,#11
H3:     MOV     R4,#248
H2:     MOV     R3,#255
H1:     DJNZ    R3,H1       ;4 MC for DS89C4x0
        DJNZ    R4,H2
        DJNZ    R5,H3
        RET
        END
```

Delay
= 11 × 248 × 255 × 4 MC × 90 ns
= 250,430 μs

# I/O PROGRAMMING

## Different ways of Accessing Entire 8 Bits

The entire 8 bits of Port 1 are accessed

```
BACK:   MOV     A,#55H
        MOV     P1,A
        ACALL   DELAY
        MOV     A,#0AAH
        MOV     P1,A
        ACALL   DELAY
        SJMP    BACK
```

Rewrite the code in a more efficient manner by accessing the port directly without going through the accumulator

```
BACK:   MOV     P1,#55H
        ACALL   DELAY
        MOV     P1,#0AAH
        ACALL   DELAY
        SJMP    BACK
```

Another way of doing the same thing

```
        MOV     A,#55H
BACK:   MOV     P1,A
        ACALL   DELAY
        CPL     A
        SJMP    BACK
```

# I/O BIT MANIPULATION PROGRAMMING

## I/O Ports and Bit Addressability

- Sometimes we need to access only 1 or 2 bits of the port

```
BACK:   CPL     P1.2            ;complement P1.2
        ACALL   DELAY
        SJMP    BACK

;another variation of the above program
AGAIN:  SETB    P1.2            ;set only P1.2
        ACALL   DELAY
        CLR     P1.2            ;clear only P1.2
        ACALL   DELAY
        SJMP    AGAIN
```

| P0 | P1 | P2 | P3 | Port Bit |
|---|---|---|---|---|
| P0.0 | P1.0 | P2.0 | P3.0 | D0 |
| P0.1 | P1.1 | P2.1 | P3.1 | D1 |
| P0.2 | P1.2 | P2.2 | P3.2 | D2 |
| P0.3 | P1.3 | P2.3 | P3.3 | D3 |
| P0.4 | P1.4 | P2.4 | P3.4 | D4 |
| P0.5 | P1.5 | P2.5 | P3.5 | D5 |
| P0.6 | P1.6 | P2.6 | P3.6 | D6 |
| P0.7 | P1.7 | P2.7 | P3.7 | D7 |

**Example 4-2**

Write the following programs.
Create a square wave of 50% duty cycle on bit 0 of port 1.

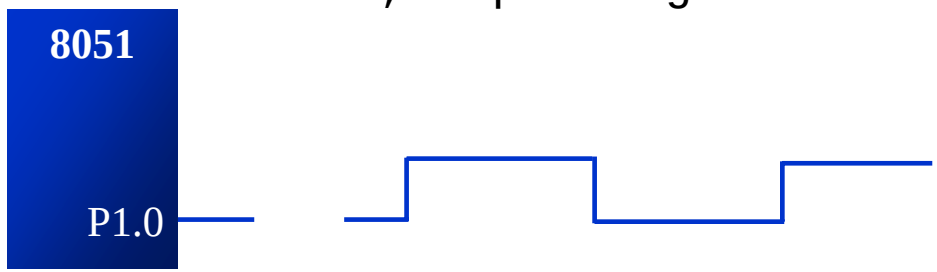
**Solution:**

The 50% duty cycle means that the "on" and "off" state (or the high and low portion of the pulse) have the same length. Therefore, we toggle P1.0 with a time delay in between each state.

```
HERE:  SETB   P1.0  ;set to high bit 0 of port 1
       LCALL  DELAY  ;call the delay subroutine
       CLR    P1.0   ;P1.0=0
       LCALL  DELAY
       SJMP   HERE   ;keep doing it
```

Another way to write the above program is:

```
HERE:  CPL    P1.0  ;set to high bit 0 of port 1
       LCALL  DELAY  ;call the delay subroutine
       SJMP   HERE   ;keep doing it
```



- Instructions that are used for signal-bit operations are as following

**Single-Bit Instructions**

| Instruction | Function |
|---|---|
| SETB bit | Set the bit (bit = 1) |
| CLR bit | Clear the bit (bit = 0) |
| CPL bit | Complement the bit (bit = NOT bit) |
| JB bit, target | Jump to target if bit = 1 (jump if bit) |
| JNB bit, target | Jump to target if bit = 0 (jump if no bit) |
| JBC bit, target | Jump to target if bit = 1, clear bit (jump if bit, then clear) |

# I/O BIT MANIPULATION PROGRAMMING

## Checking an Input Bit

- The JNB and JB instructions are widely used single-bit operations
  - They allow you to monitor a bit and make a decision depending on whether it's 0 or 1
  - These two instructions can be used for any bits of I/O ports 0, 1, 2, and 3
    - Port 3 is typically not used for any I/O, either single-bit or byte-wise

**Instructions for Reading an Input Port**

| Mnemonic | Examples | Description |
|---|---|---|
| MOV A,PX | MOV A,P2 | Bring into A the data at P2 pins |
| JNB PX.Y, .. | JNB P2.1,TARGET | Jump if pin P2.1 is low |
| JB PX.Y, .. | JB P1.3,TARGET | Jump if pin P1.3 is high |
| MOV C,PX.Y | MOV C,P2.4 | Copy status of pin P2.4 to CY |

# I/O BIT MANIPULATION PROGRAMMING

## Checking an Input Bit (cont')

**Example 4-3**

Write a program to perform the following:
(a) Keep monitoring the P1.2 bit until it becomes high
(b) When P1.2 becomes high, write value 45H to port 0
(c) Send a high-to-low (H-to-L) pulse to P2.3

**Solution:**

```
        SETB P1.2        ;make P1.2 an input
        MOV  A,#45H      ;A=45H
AGAIN:  JNB  P1.2,AGAIN  ; get out when P1.2=1
        MOV  P0,A        ;issue A to P0
        SETB P2.3        ;make P2.3 high
        CLR  P2.3        ;make P2.3 low for H-to-L
```

# I/O BIT MANIPULATION PROGRAMMING

## Checking an Input Bit (cont')

**Example 4-4**

Assume that bit P2.3 is an input and represents the condition of an oven. If it goes high, it means that the oven is hot. Monitor the bit continuously. Whenever it goes high, send a high-to-low pulse to port P1.5 to turn on a buzzer.

**Solution:**

```
HERE:  JNB  P2.3,HERE ;keep monitoring for high
       SETB P1.5      ;set bit P1.5=1
       CLR  P1.5      ;make high-to-low
       SJMP HERE      ;keep repeating
```

# I/O BIT MANIPULATION PROGRAMMING

## Checking an Input Bit (cont')

**Example 4-5**

A switch is connected to pin P1.7. Write a program to check the status of SW and perform the following:
(a) If SW=0, send letter 'N' to P2
(b) If SW=1, send letter 'Y' to P2

**Solution:**

```
        SETB P1.7        ;make P1.7 an input
AGAIN:  JB   P1.2,OVER   ;jump if P1.7=1
        MOV  P2,#'N'     ;SW=0, issue 'N' to P2
        SJMP AGAIN       ;keep monitoring
OVER:   MOV  P2,#'Y'     ;SW=1, issue 'Y' to P2
        SJMP AGAIN       ;keep monitoring
```

# I/O BIT MANIPULATION PROGRAMMING

## Reading Single Bit into Carry Flag

**Example 4-6**

A switch is connected to pin P1.7. Write a program to check the status of SW and perform the following:
(a) If SW=0, send letter 'N' to P2
(b) If SW=1, send letter 'Y' to P2
Use the carry flag to check the switch status.

**Solution:**

```
        SETB P1.7       ;make P1.7 an input
AGAIN:  MOV  C,P1.2     ;read SW status into CF
        JC   OVER       ;jump if SW=1
        MOV  P2,#'N'    ;SW=0, issue 'N' to P2
        SJMP AGAIN      ;keep monitoring
OVER:   MOV  P2,#'Y'    ;SW=1, issue 'Y' to P2
        SJMP AGAIN      ;keep monitoring
```

# I/O BIT MANIPULATION PROGRAMMING

## Reading Single Bit into Carry Flag (cont')

**Example 4-7**

A switch is connected to pin P1.0 and an LED to pin P2.7. Write a program to get the status of the switch and send it to the LED

**Solution:**

```
        SETB P1.7       ;make P1.7 an input
AGAIN:  MOV  C,P1.0     ;read SW status into CF
        MOV  P2.7,C     ;send SW status to LED
        SJMP AGAIN      ;keep repeating
```

The instruction 'MOV P2.7,P1.0' is wrong , since such an instruction does not exist

However 'MOV P2,P1' is a valid instruction

- In reading a port
  - Some instructions read the status of port pins
  - Others read the status of an internal port latch
- Therefore, when reading ports there are two possibilities:
  - Read the status of the input pin
  - Read the internal latch of the output port
- Confusion between them is a major source of errors in 8051 programming
  - Especially where external hardware is concerned

## READING INPUT PINS VS. PORT LATCH

## Reading Latch for Output Port

- Some instructions read the contents of an internal port latch instead of reading the status of an external pin
  - For example, look at the `ANL P1,A` instruction and the sequence of actions is executed as follow
    1. It reads the internal latch of the port and brings that data into the CPU
    2. This data is ANDed with the contents of register A
    3. The result is rewritten back to the port latch
    4. The port pin data is changed and now has the same value as port latch

READING INPUT PINS VS. PORT LATCH

Reading Latch for Output Port (cont')

- *Read-Modify-Write*
  - The instructions read the port latch normally read a value, perform an operation then rewrite it back to the port latch

**Instructions Reading a latch (Read-Modify-Write)**

| Mnemonics | Example |
|---|---|
| ANL PX | ANL P1,A |
| ORL PX | ORL P2,A |
| XRL PX | XRL P0,A |
| JBC PX.Y,TARGET | JBC P1.1,TARGET |
| CPL PX.Y | CPL P1.2 |
| INC PX | INC P1 |
| DEC PX | DEC P2 |
| DJNZ PX.Y,TARGET | DJNZ P1,TARGET |
| MOV PX.Y,C | MOV P1.2,C |
| CLR PX.Y | CLR P2.3 |
| SETB PX.Y | SETB P2.3 |

Note: x is 0, 1, 2, or 3 for P0 – P3

# I/O BIT MANIPULATION PROGRAMMING

## Read-modify-write Feature

- The ports in 8051 can be accessed by the Read-modify-write technique
  - This feature saves many lines of code by combining in a single instruction all three actions
    1. Reading the port
    2. Modifying it
    3. Writing to the port

```
        MOV    P1,#55H   ;P1=01010101
AGAIN:  XRL    P1,#0FFH  ;EX-OR P1 with 1111 1111
        ACALL  DELAY
        SJMP   BACK
```

# ADDRESSING MODES

*The 8051 Microcontroller and Embedded Systems: Using Assembly and C*
Mazidi, Mazidi and McKinlay

Chung-Ping Young
楊中平



***Home Automation, Networking, and Entertainment Lab***

Dept. of Computer Science and Information Engineering
National Cheng Kung University, TAIWAN

# ADDRESSING MODES

- The CPU can access data in various ways, which are called *addressing modes*
  - Immediate
  - Register
  - Direct
  - Register indirect
  - Indexed

Accessing memories

# IMMEDIATE ADDRESSING MODE

- The source operand is a constant
  - The immediate data must be preceded by the pound sign, “#”
  - Can load information into any registers, including 16-bit DPTR register
    - DPTR can also be accessed as two 8-bit registers, the high byte DPH and low byte DPL

```
MOV A,#25H       ;load 25H into A
MOV R4,#62       ;load 62 into R4
MOV B,#40H       ;load 40H into B
MOV DPTR,#4521H ;DPTR=4512H
MOV DPL,#21H     ;This is the same
MOV DPH,#45H     ;as above

;illegal!! Value > 65535 (FFFFH)
MOV DPTR,#68975
```

# IMMEDIATE ADDRESSING MODE (cont')

- We can use EQU directive to access immediate data

```
        Count   EQU 30
        ...     ...
        MOV     R4,#COUNT          ;R4=1EH
        MOV     DPTR,#MYDATA       ;DPTR=200H

        ORG     200H
MYDATA: DB      "America"
```

- We can also use immediate addressing mode to send data to 8051 ports

```
        MOV P1,#55H
```

# REGISTER ADDRESSING MODE

- Use registers to hold the data to be manipulated

```
MOV A,R0    ;copy contents of R0 into A
MOV R2,A    ;copy contents of A into R2
ADD A,R5    ;add contents of R5 to A
ADD A,R7    ;add contents of R7 to A
MOV R6,A    ;save accumulator in R6
```

- The source and destination registers must match in size
  - `MOV DPTR,A` will give an error

```
MOV DPTR,#25F5H
MOV R7,DPL
MOV R6,DPH
```

- The movement of data between Rn registers is not allowed
  - `MOV R4,R7` is invalid

ACCESSING MEMORY

Direct Addressing Mode

- It is most often used the direct addressing mode to access RAM locations 30 – 7FH
  - The entire 128 bytes of RAM can be accessed
  - The register bank locations are accessed by the register names

Direct addressing mode

```
MOV A,4     ;is same as
MOV A,R4    ;which means copy R4 into A
```

- Contrast this with immediate addressing mode
  - There is no "#" sign in the operand

Register addressing mode

```
MOV R0,40H  ;save content of 40H in R0
MOV 56H,A   ;save content of A in 56H
```

ACCESSING MEMORY

SFR Registers and Their Addresses

- The SFR (*Special Function Register*) can be accessed by their names or by their addresses

```
MOV 0E0H,#55H    ;is the same as
MOV A,#55h       ;load 55H into A

MOV 0F0H,R0      ;is the same as
MOV B,R0         ;copy R0 into B
```

- The SFR registers have addresses between 80H and FFH
  - Not all the address space of 80 to FF is used by SFR
  - The unused locations 80H to FFH are reserved and must not be used by the 8051 programmer

# ACCESSING MEMORY

## SFR Registers and Their Addresses (cont')

### Special Function Register (SFR) Addresses

| Symbol | Name | Address |
|---|---|---|
| ACC* | Accumulator | 0E0H |
| B* | B register | 0F0H |
| PSW* | Program status word | 0D0H |
| SP | Stack pointer | 81H |
| DPTR | Data pointer 2 bytes | |
| DPL | Low byte | 82H |
| DPH | High byte | 83H |
| P0* | Port 0 | 80H |
| P1* | Port 1 | 90H |
| P2* | Port 2 | 0A0H |
| P3* | Port 3 | 0B0H |
| IP* | Interrupt priority control | 0B8H |
| IE* | Interrupt enable control | 0A8H |
| ... | ... | ... |

ACCESSING MEMORY

SFR Registers and Their Addresses (cont')

## Special Function Register (SFR) Addresses

| Symbol | Name | Address |
|---|---|---|
| TMOD | Timer/counter mode control | 89H |
| TCON* | Timer/counter control | 88H |
| T2CON* | Timer/counter 2 control | 0C8H |
| T2MOD | Timer/counter mode control | OC9H |
| TH0 | Timer/counter 0 high byte | 8CH |
| TL0 | Timer/counter 0 low byte | 8AH |
| TH1 | Timer/counter 1 high byte | 8DH |
| TL1 | Timer/counter 1 low byte | 8BH |
| TH2 | Timer/counter 2 high byte | 0CDH |
| TL2 | Timer/counter 2 low byte | 0CCH |
| RCAP2H | T/C 2 capture register high byte | 0CBH |
| RCAP2L | T/C 2 capture register low byte | 0CAH |
| SCON* | Serial control | 98H |
| SBUF | Serial data buffer | 99H |
| PCON | Power ontrol | 87H |

*** Bit addressable**

# ACCESSING MEMORY

## SFR Registers and Their Addresses (cont')

**Example 5-1**

Write code to send 55H to ports P1 and P2, using

(a) their names (b) their addresses

**Solution :**

```
(a)   MOV A,#55H        ;A=55H
      MOV P1,A          ;P1=55H
      MOV P2,A          ;P2=55H
```

(b) From Table 5-1, P1 address=80H; P2 address=A0H

```
      MOV A,#55H        ;A=55H
      MOV 80H,A         ;P1=55H
      MOV 0A0H,A        ;P2=55H
```

ACCESSING MEMORY

Stack and Direct Addressing Mode

- Only direct addressing mode is allowed for pushing or popping the stack
  - PUSH A is invalid
  - Pushing the accumulator onto the stack must be coded as `PUSH 0E0H`

**Example 5-2**

Show the code to push R5 and A onto the stack and then pop them back them into R2 and B, where B = A and R2 = R5

**Solution:**

```
PUSH 05        ;push R5 onto stack
PUSH 0E0H      ;push register A onto stack
POP  0F0H      ;pop top of stack into B
               ;now register B = register A
POP  02        ;pop top of stack into R2
               ;now R2=R6
```

ACCESSING MEMORY

Register Indirect Addressing Mode

- A register is used as a pointer to the data
  - Only register R0 and R1 are used for this purpose
  - R2 – R7 cannot be used to hold the address of an operand located in RAM
- When R0 and R1 hold the addresses of RAM locations, they must be preceded by the “@” sign

```
        MOV A,@R0   ;move contents of RAM whose
                    ;address is held by R0 into A
        MOV @R1,B   ;move contents of B into RAM
                    ;whose address is held by R1
```

ACCESSING MEMORY

Register Indirect Addressing Mode (cont')

**Example 5-3**

Write a program to copy the value 55H into RAM memory locations 40H to 41H using
(a) direct addressing mode, (b) register indirect addressing mode without a loop, and (c) with a loop

**Solution:**

```
(a)
    MOV A,#55H  ;load A with value 55H
    MOV 40H,A   ;copy A to RAM location 40H
    MOV 41H.A   ;copy A to RAM location 41H
(b)
    MOV A,#55H  ;load A with value 55H
    MOV R0,#40H ;load the pointer. R0=40H
    MOV @R0,A   ;copy A to RAM R0 points to
    INC R0      ;increment pointer. Now R0=41h
    MOV @R0,A   ;copy A to RAM R0 points to
(c)
        MOV A,#55H    ;A=55H
        MOV R0,#40H   ;load pointer.R0=40H,
        MOV R2,#02    ;load counter, R2=3
AGAIN:  MOV @R0,A     ;copy 55 to RAM R0 points to
        INC R0        ;increment R0 pointer
        DJNZ R2,AGAIN ;loop until counter = zero
```

ACCESSING MEMORY

Register Indirect Addressing Mode (cont')

- The advantage is that it makes accessing data dynamic rather than static as in direct addressing mode
  - Looping is not possible in direct addressing mode

**Example 5-4**

Write a program to clear 16 RAM locations starting at RAM address 60H

**Solution:**

```
        CLR A          ;A=0
        MOV R1,#60H    ;load pointer. R1=60H
        MOV R7,#16     ;load counter, R7=16
AGAIN:  MOV @R1,A      ;clear RAM R1 points to
        INC R1         ;increment R1 pointer
        DJNZ R7,AGAIN  ;loop until counter=zero
```

ACCESSING MEMORY

Register Indirect Addressing Mode (cont')

**Example 5-5**

Write a program to copy a block of 10 bytes of data from 35H to 60H

**Solution:**

```
       MOV R0,#35H   ;source pointer
       MOV R1,#60H   ;destination pointer
       MOV R3,#10    ;counter
BACK:  MOV A,@R0     ;get a byte from source
       MOV @R1,A     ;copy it to destination
       INC R0        ;increment source pointer
       INC R1      ;increment destination pointer
       DJNZ R3,BACK ;keep doing for ten bytes
```



## ACCESSING MEMORY

## Register Indirect Addressing Mode (cont')

- R0 and R1 are the only registers that can be used for pointers in register indirect addressing mode
- Since R0 and R1 are 8 bits wide, their use is limited to access any information in the internal RAM
- Whether accessing externally connected RAM or on-chip ROM, we need 16-bit pointer
  - In such case, the DPTR register is used

ACCESSING MEMORY

Indexed Addressing Mode and On-chip ROM Access

- Indexed addressing mode is widely used in accessing data elements of look-up table entries located in the program ROM
- The instruction used for this purpose is `MOVC A,@A+DPTR`
  - Use instruction `MOVC`, “C” means code
  - The contents of A are added to the 16-bit register DPTR to form the 16-bit address of the needed data

DPTR=200H, A=0

DPTR=200H, A=55H

DPTR=201H, A=55H

DPTR=201H, A=0

DPTR=201H, A=53H

DPTR=202H, A=53H

| | |
|---|---|
| 202 | A |
| 201 | S |
| 200 | U |

**Example 5-6**

In this program, assume that the word "USA" is burned into ROM locations starting at 200H. And that the program is burned into ROM locations starting at 0. Analyze how the program works and state where "USA" is stored after this program is run.

**Solution:**

```
        ORG   0000H        ;burn into ROM starting at 0
        MOV   DPTR,#200H   ;DPTR=200H look-up table addr
        CLR   A            ;clear A(A=0)
        MOVC  A,@A+DPTR    ;get the char from code space
        MOV   R0,A         ;save it in R0
        INC   DPTR         ;DPTR=201 point to next char
        CLR   A            ;clear A(A=0)
        MOVC  A,@A+DPTR    ;get the next char
        MOV   R1,A         ;save it in R1
        INC   DPTR         ;DPTR=202 point to next char
        CLR   A            ;clear A(A=0)
        MOVC  A,@A+DPTR    ;get the next char
        MOV   R2,A         ;save it in R2
Here:   SJMP  HERE         ;stay here
;Data is burned into code space starting at 200H

        ORG   200H
MYDATA:DB  "USA"
        END                ;end of program
```

R0=55H

R1=53H

R2=41H

- The look-up table allows access to elements of a frequently used table with minimum operations

**Example 5-8**

Write a program to get the x value from P1 and send $x^2$ to P2, continuously

**Solution:**

```
      ORG   0
      MOV   DPTR,#300H    ;LOAD TABLE ADDRESS
      MOV   A,#0FFH       ;A=FF
      MOV   P1,A          ;CONFIGURE P1 INPUT PORT
BACK:MOV    A,P1          ;GET X
      MOV   A,@A+DPTR     ;GET X SQAURE FROM TABLE
      MOV   P2,A          ;ISSUE IT TO P2
      SJMP  BACK          ;KEEP DOING IT

      ORG   300H
XSQR_TABLE:
      DB    0,1,4,9,16,25,36,49,64,81
      END
```

ACCESSING MEMORY

Indexed Addressing Mode and MOVX

- In many applications, the size of program code does not leave any room to share the 64K-byte code space with data
  - The 8051 has another 64K bytes of memory space set aside exclusively for data storage
    - This data memory space is referred to as *external memory* and it is accessed only by the `MOVX` instruction
- The 8051 has a total of 128K bytes of memory space
  - 64K bytes of code and 64K bytes of data
  - The data space cannot be shared between code and data

ACCESSING MEMORY

RAM Locations 30 – 7FH as Scratch Pad

- In many applications we use RAM locations 30 – 7FH as scratch pad
  - We use R0 – R7 of bank 0
  - Leave addresses 8 – 1FH for stack usage
  - If we need more registers, we simply use RAM locations 30 – 7FH

**Example 5-10**

Write a program to toggle P1 a total of 200 times. Use RAM location 32H to hold your counter value instead of registers R0 – R7

**Solution:**

```
        MOV    P1,#55H    ;P1=55H
        MOV    32H,#200   ;load counter value
                          ;into RAM loc 32H
LOP1:   CPL    P1         ;toggle P1
        ACALL  DELAY
        DJNZ   32H,LOP1   ;repeat 200 times
```

# BIT ADDRESSES

- Many microprocessors allow program to access registers and I/O ports in byte size only
  - However, in many applications we need to check a single bit
- One unique and powerful feature of the 8051 is single-bit operation
  - Single-bit instructions allow the programmer to set, clear, move, and complement individual bits of a port, memory, or register
  - It is registers, RAM, and I/O ports that need to be bit-addressable
    - ROM, holding program code for execution, is not bit-addressable

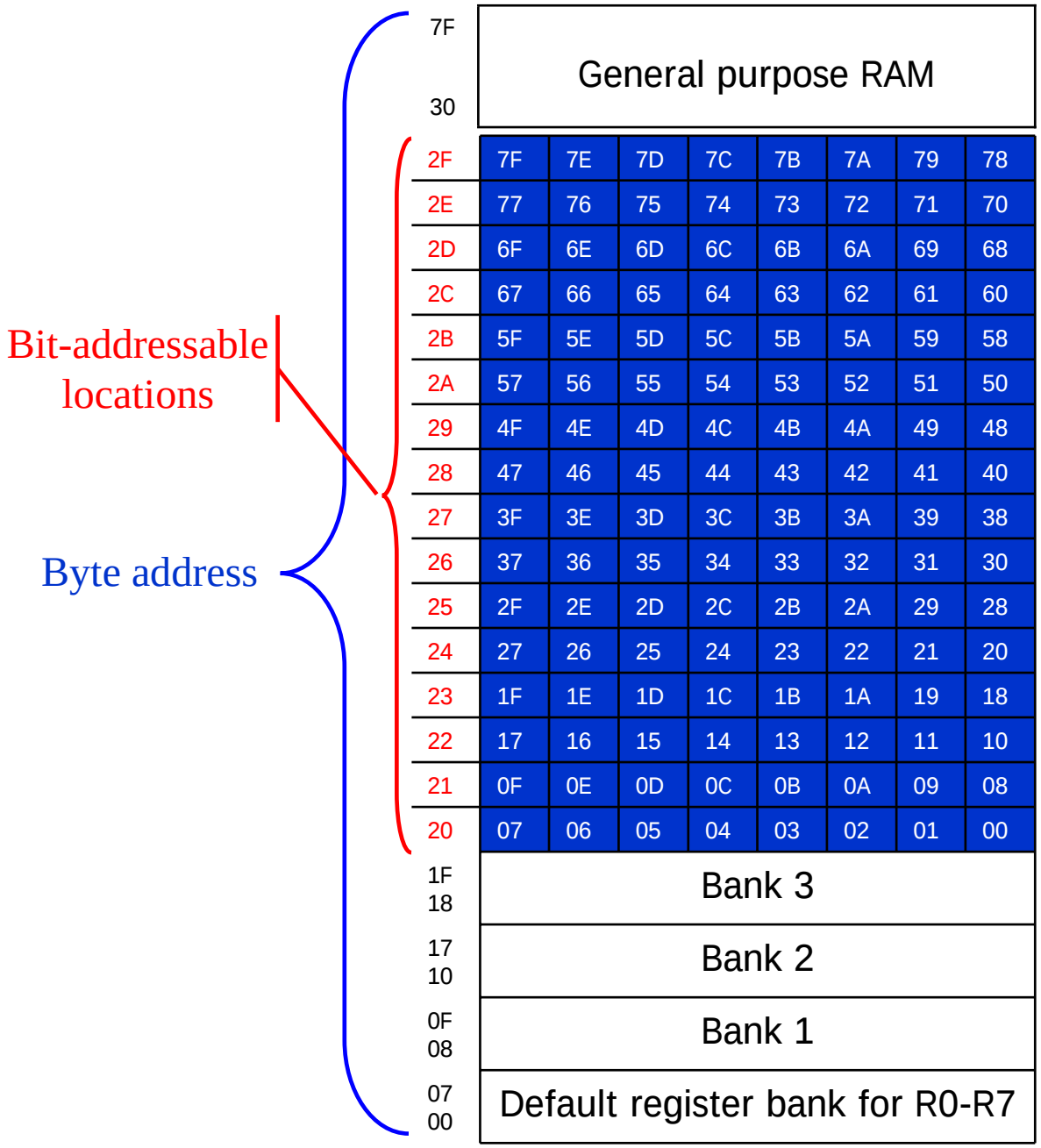
# BIT ADDRESSES

## Bit-Addressable RAM

- The bit-addressable RAM location are 20H to 2FH
  - These 16 bytes provide 128 bits of RAM bit-addressability, since 16 × 8 = 128
    - 0 to 127 (in decimal) or 00 to 7FH
  - The first byte of internal RAM location 20H has bit address 0 to 7H
  - The last byte of 2FH has bit address 78H to 7FH
- Internal RAM locations 20-2FH are both byte-addressable and bit-addressable
  - Bit address 00-7FH belong to RAM byte addresses 20-2FH
  - Bit address 80-F7H belong to SFR P0, P1, …

# BIT ADDRESSES

## Bit-Addressable RAM (cont')



# BIT ADDRESSES

## Bit-Addressable RAM (cont')

**Example 5-11**

Find out to which by each of the following bits belongs. Give the address of the RAM byte in hex

(a) `SETB 42H`, (b) `CLR 67H`, (c) `CLR 0FH`
(d) `SETB 28H`, (e) `CLR 12`, (f) `SETB 05`

**Solution:**

(a) D2 of RAM location 28H

(b) D7 of RAM location 2CH

(c) D7 of RAM location 21H

(d) D0 of RAM location 25H

(e) D4 of RAM location 21H

(f) D5 of RAM location 20H

| | D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|---|---|---|---|---|---|---|---|---|
| 2F | 7F | 7E | 7D | 7C | 7B | 7A | 79 | 78 |
| 2E | 77 | 76 | 75 | 74 | 73 | 72 | 71 | 70 |
| 2D | 6F | 6E | 6D | 6C | 6B | 6A | 69 | 68 |
| 2C | 67 | 66 | 65 | 64 | 63 | 62 | 61 | 60 |
| 2B | 5F | 5E | 5D | 5C | 5B | 5A | 59 | 58 |
| 2A | 57 | 56 | 55 | 54 | 53 | 52 | 51 | 50 |
| 29 | 4F | 4E | 4D | 4C | 4B | 4A | 49 | 48 |
| 28 | 47 | 46 | 45 | 44 | 43 | 42 | 41 | 40 |
| 27 | 3F | 3E | 3D | 3C | 3B | 3A | 39 | 38 |
| 26 | 37 | 36 | 35 | 34 | 33 | 32 | 31 | 30 |
| 25 | 2F | 2E | 2D | 2C | 2B | 2A | 29 | 28 |
| 24 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 |
| 23 | 1F | 1E | 1D | 1C | 1B | 1A | 19 | 18 |
| 22 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 |
| 21 | 0F | 0E | 0D | 0C | 0B | 0A | 09 | 08 |
| 20 | 07 | 06 | 05 | 04 | 03 | 02 | 01 | 00 |

# BIT ADDRESSES

## Bit-Addressable RAM (cont')

- To avoid confusion regarding the addresses 00 – 7FH
  - The 128 bytes of RAM have the byte addresses of 00 – 7FH can be accessed in byte size using various addressing modes
    - Direct and register-indirect
  - The 16 bytes of RAM locations 20 – 2FH have bit address of 00 – 7FH
    - We can use only the single-bit instructions and these instructions use only direct addressing mode

BIT ADDRESSES

Bit-Addressable RAM (cont')

- Instructions that are used for signal-bit operations are as following

**Single-Bit Instructions**

| Instruction | Function |
|---|---|
| SETB bit | Set the bit (bit = 1) |
| CLR bit | Clear the bit (bit = 0) |
| CPL bit | Complement the bit (bit = NOT bit) |
| JB bit, target | Jump to target if bit = 1 (jump if bit) |
| JNB bit, target | Jump to target if bit = 0 (jump if no bit) |
| JBC bit, target | Jump to target if bit = 1, clear bit (jump if bit, then clear) |

# BIT ADDRESSES

## I/O Port Bit Addresses

- While all of the SFR registers are byte-addressable, some of them are also bit-addressable
  - The P0 – P3 are bit addressable
- We can access either the entire 8 bits or any single bit of I/O ports P0, P1, P2, and P3 without altering the rest
- When accessing a port in a single-bit manner, we use the syntax `SETB X.Y`
  - X is the port number P0, P1, P2, or P3
  - Y is the desired bit number from 0 to 7 for data bits D0 to D7
  - ex. `SETB P1.5` sets bit 5 of port 1 high

# BIT ADDRESSES

## I/O Port Bit Addresses (cont')

- Notice that when code such as `SETB P1.0` is assembled, it becomes `SETB 90H`
  - The bit address for I/O ports
    - P0 are 80H to 87H
    - P1 are 90H to 97H
    - P2 are A0H to A7H
    - P3 are B0H to B7H

**Single-Bit Addressability of Ports**

| P0 | P1 | P2 | P3 | Port Bit |
|---|---|---|---|---|
| P0.0 (80) | P1.0 (90) | P2.0 (A0) | P3.0 (B0) | D0 |
| P0.1 | P1.1 | P2.1 | P3.1 | D1 |
| P0.2 | P1.2 | P2.2 | P3.2 | D2 |
| P0.3 | P1.3 | P2.3 | P3.3 | D3 |
| P0.4 | P1.4 | P2.4 | P3.4 | D4 |
| P0.5 | P1.5 | P2.5 | P3.5 | D5 |
| P0.6 | P1.6 | P2.6 | P3.6 | D6 |
| P0.7 (87) | P1.7 (97) | P2.7 (A7) | P3.7 (B7) | D7 |

# BIT ADDRESSES

## I/O Port Bit Addresses (cont')

## SFR RAM Address (Byte and Bit)

Bit addresses 80 – F7H belong to SFR of P0, TCON, P1, SCON, P2, etc

| Byte address | Bit address | |
|---|---|---|
| FF | | |
| F0 | F7 F6 F5 F4 F3 F2 F1 F0 | B |
| E0 | E7 E6 E5 E4 E3 E2 E1 E0 | ACC |
| D0 | D7 D6 D5 D4 D3 D2 D1 D0 | PSW |
| B8 | -- -- -- BC BB BA B9 B8 | IP |
| B0 | B7 B6 B5 B4 B3 B2 B1 B0 | P3 |
| A8 | AF AE AD AC AB AA A9 A8 | IE |
| A0 | A7 A6 A5 A4 A3 A2 A1 A0 | P2 |
| 99 | not bit addressable | SBUF |

| Byte address | Bit address | |
|---|---|---|
| 98 | 9F 9E 9D 9C 9B 9A 99 98 | SCON |
| 90 | 97 96 95 94 93 92 91 90 | P1 |
| 8D | not bit addressable | TH1 |
| 8C | not bit addressable | TH0 |
| 8B | not bit addressable | TL1 |
| 8A | not bit addressable | TL0 |
| 89 | not bit addressable | TMOD |
| 88 | 8F 8E 8D 8C 8B 8A 89 88 | TCON |
| 87 | not bit addressable | PCON |
| 83 | not bit addressable | DPH |
| 82 | not bit addressable | DPL |
| 81 | not bit addressable | SP |
| 80 | 87 86 85 84 83 82 81 80 | P0 |

Special Function Register

# BIT ADDRESSES

## Registers Bit-Addressability

- Only registers A, B, PSW, IP, IE, ACC, SCON, and TCON are bit-addressable
  - While all I/O ports are bit-addressable
- In PSW register, two bits are set aside for the selection of the register banks
  - Upon RESET, bank 0 is selected
  - We can select any other banks using the bit-addressability of the PSW

| CY | AC | -- | RS1 | RS0 | OV | -- | P |
|---|---|---|---|---|---|---|---|

| RS1 | RS0 | Register Bank | Address |
|---|---|---|---|
| 0 | 0 | 0 | 00H - 07H |
| 0 | 1 | 1 | 08H - 0FH |
| 1 | 0 | 2 | 10H - 17H |
| 1 | 1 | 3 | 18H - 1FH |

# BIT ADDRESSES

## Registers Bit-Addressability (cont')

**Example 5-13**

Write a program to save the accumulator in R7 of bank 2.

**Solution:**

```
        CLR     PSW.3
        SETB    PSW.4
        MOV     R7,A
```

**Example 5-14**

While there are instructions such as JNC and JC to check the carry flag bit (CY), there are no such instructions for the overflow flag bit (OV). How would you write code to check OV?

**Solution:**

```
        JB      PSW.2,TARGET    ;jump if OV=1
```




**Example 5-18**

While a program to save the status of bit P1.7 on RAM address bit 05.

**Solution:**

```
        MOV     C,P1.7
        MOV     05,C
```

# BIT ADDRESSES

## Registers Bit-Addressability (cont')

**Example 5-15**

Write a program to see if the RAM location 37H contains an even value. If so, send it to P2. If not, make it even and then send it to P2.

**Solution:**

```
        MOV     A,37H       ;load RAM 37H into ACC
        JNB     ACC.0,YES   ;if D0 of ACC 0? If so jump
        INC     A           ;it's odd, make it even
YES:    MOV     P2,A        ;send it to P2
```

**Example 5-17**

The status of bits P1.2 and P1.3 of I/O port P1 must be saved before they are changed. Write a program to save the status of P1.2 in bit location 06 and the status of P1.3 in bit location 07

**Solution:**

```
        CLR     06          ;clear bit addr. 06
        CLR     07          ;clear bit addr. 07
        JNB     P1.2,OVER   ;check P1.2, if 0 then jump
        SETB    06          ;if P1.2=1,set bit 06 to 1
OVER:   JNB     P1.3,NEXT   ;check P1.3, if 0 then jump
        SETB    07          ;if P1.3=1,set bit 07 to 1
NEXT:   ...
```

## BIT ADDRESSES

## Using BIT

- The `BIT` directive is a widely used directive to assign the bit-addressable I/O and RAM locations
  - Allow a program to assign the I/O or RAM bit at the beginning of the program, making it easier to modify them

**Example 5-22**

A switch is connected to pin P1.7 and an LED to pin P2.0. Write a program to get the status of the switch and send it to the LED.

**Solution:**

```
LED     BIT     P1.7    ;assign bit
SW      BIT     P2.0    ;assign bit
HERE:   MOV     C,SW    ;get the bit from the port
        MOV     LED,C   ;send the bit to the port
        SJMP    HERE    ;repeat forever
```